



第7章 图

7.1 图的定义和术语

7.2 图的存储结构

7.3 图的遍历

7.4 最小生成树

7.5 活动网络

7.6 最短路径

7.1 图的定义和术语

□ **图(Graph)**——图**G**是由两个集合**V(G)**和**E(G)**组成的,记为

$$G = (V, E)$$

V=vertex
E=edge

其中: **V(G)**是**顶点**的非空有限集

E(G)是**边**的有限集合, 边是顶点的**无序对**或**有序对**。

$E = \{(x, y) \mid x, y \in V \ \&\& \text{Path}(x, y)\}$ 无向图
或

$E = \{\langle x, y \rangle \mid x, y \in V \ \&\& \text{Path}\langle x, y \rangle\}$ 有向图

$\text{Path}\langle x, y \rangle$ 表示从 x 到 y 的一条单向通路, 它是有方向的。 x 是弧尾, y 是弧头。

7.1 图的定义和术语

- **有向图**——有向图 G 是由两个集合 $V(G)$ 和 $E(G)$ 组成的

其中： $V(G)$ 是顶点的非空有限集

$E(G)$ 是有向边（也称弧）的有限集合，弧是顶点的有序对，记为 $\langle v, w \rangle$ ， v, w 是顶点， v 为弧尾， w 为弧头

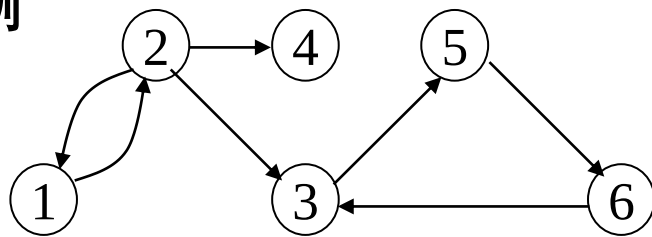
- **无向图**——无向图 G 是由两个集合 $V(G)$ 和 $E(G)$ 组成的

其中： $V(G)$ 是顶点的非空有限集

$E(G)$ 是边的有限集合，边是顶点的无序对，记为 (v, w) 或 (w, v) ，并且 $(v, w) = (w, v)$

7.1 图的定义和术语

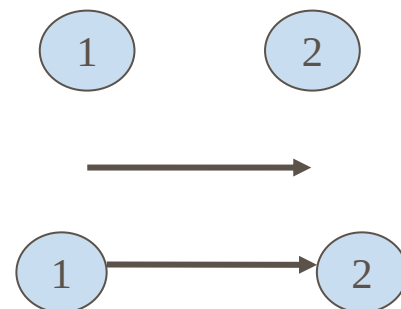
例



G1

结点(顶点)

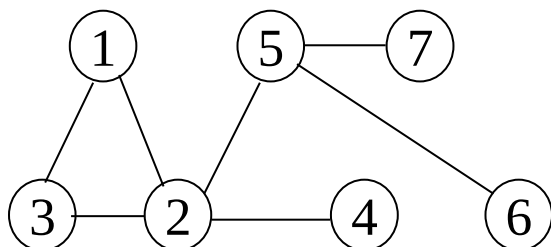
有向边(弧)、
弧尾(初始结点)、
弧头(终止结点)



有向图G1= (V, E) 中: $V(G1)=\{1,2,3,4,5,6\}$

$E(G1)=\{<1,2>, <2,1>, <2,3>, <2,4>, <3,5>, <5,6>, <6,3>\}$

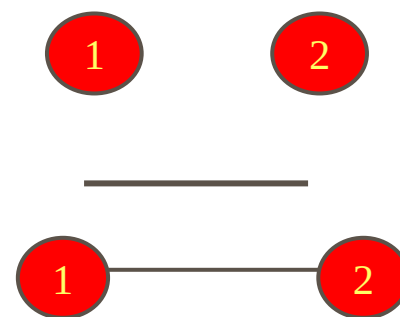
例



G2

结点(顶点)

边(无向边)



无向图G2= (V, E) 中: $V(G2)=\{1,2,3,4,5,6,7\}$

$E(G2)=\{(1,2), (1,3), (2,3), (2,4), (2,5), (5,6), (5,7)\}$

7.1 图的定义和术语

□ **完全图**——图**G**任意两个顶点都有一条边相连接

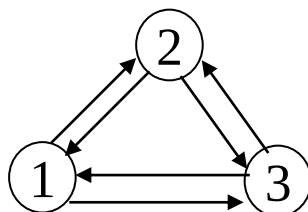
无向完全图：在具有**n** 个顶点的有向图中，

最大弧数为 $n(n-1)$

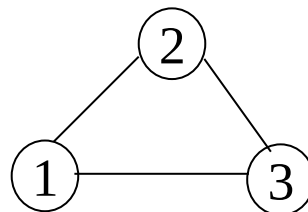
有向完全图：在具有**n** 个顶点的无向图中，

最大边数为 $n(n-1)/2$

例



有向完全图



无向完全图

7.1 图的定义和术语

证明:

① 完全有向图有 $n(n-1)$ 条边。

证明: 若是完全有向图, 则顶点1必与所有其他顶点各有2条连线, 即有 $2(n-1)$ 条边, 顶点2有 $2(n-2)$ 条边, ..., 顶点 $n-1$ 有2条边, 顶点 n 有0条边.

总边数 = $2(n-1 + n-2 + \dots + 1 + 0) = 2(n-1+0)n/2 = n(n-1)$

② 完全无向图有 $n(n-1)/2$ 条边。

证明: 若是完全无向图, 则顶点1必与所有其他顶点各有1条连线, 即有 $n-1$ 条边, 顶点2有 $n-2$ 条边, ..., 顶点 $n-1$ 有1条边, 顶点 n 有0条边.

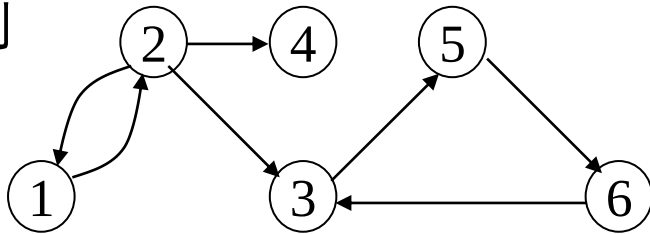
总边数 = $n-1 + n-2 + \dots + 1 + 0 = (n-1+0)n/2 = n(n-1)/2$

7.1 图的定义和术语

- 稀疏图--若边或弧的个数 $e < n \log n$ ，则称作稀疏图，否则成稠密图。
- 权—把图的边或弧赋予一个有意义的数，此数叫权
- 带权图-网—弧或边带权的图分别称作有向网或无向网。
- 子图——如果图 $G(V, E)$ 和图 $G'(V', E')$ ，满足：
 $V' \subseteq V$ 且 $E' \subseteq E$ ，则称 G' 为 G 的子图
- 邻接点—若无向图 G 中存在 (V, W) ，则称 V, W 互为邻接点；
边 (V, W) 和顶点 V, W 相关联。

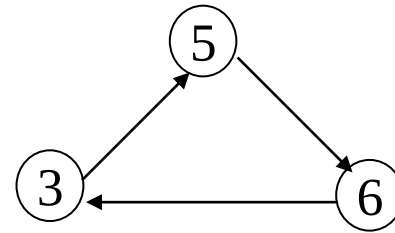
7.1 图的定义和术语

例

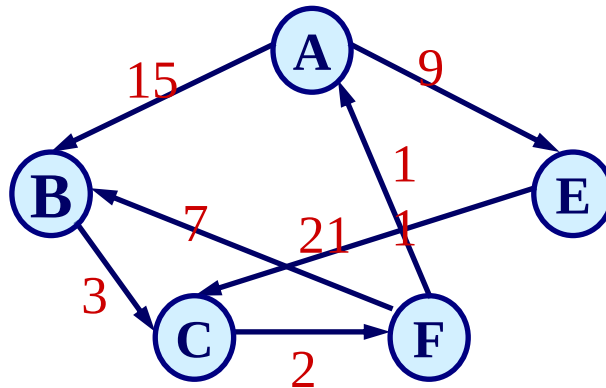


$G = (V, E)$

图G与子图G'



$G' = (V', E')$



有向网 (弧带权值)

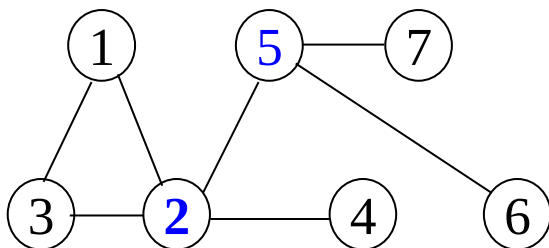
7.1 图的定义和术语

□ 顶点的度

- 无向图中，顶点的度为与该顶点相连的边数
- 有向图中，顶点的度分成入度与出度
 - 入度：以该顶点为弧头的弧的数目
 - 出度：以该顶点为弧尾的弧的数目

7.1 图的定义和术语

例

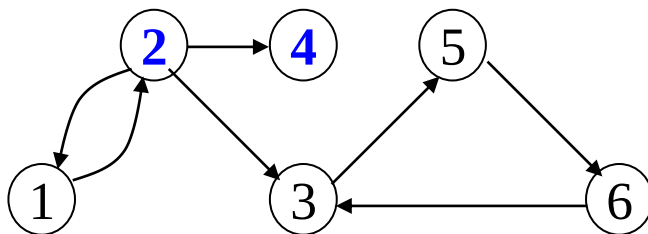


G2

顶点5的度: 3

顶点2的度: 4

例



G1

顶点2入度: 1

出度: 3

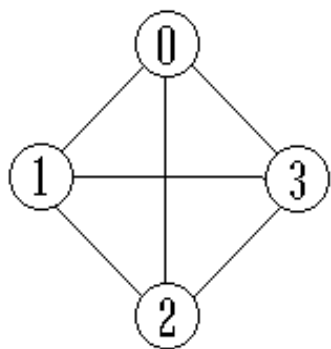
顶点4入度: 1

出度: 0

有向图顶点的度(TD)=出度(OD)+入度(ID)

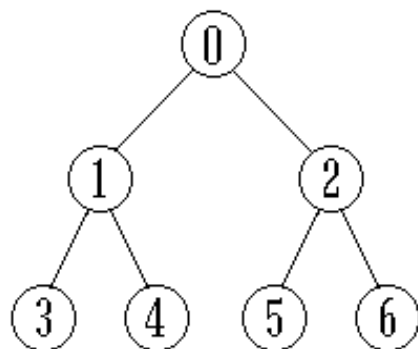
7.1 图的定义和术语

例：判断下列4种图形各属什么类型？



(a) G1

无向 完全图



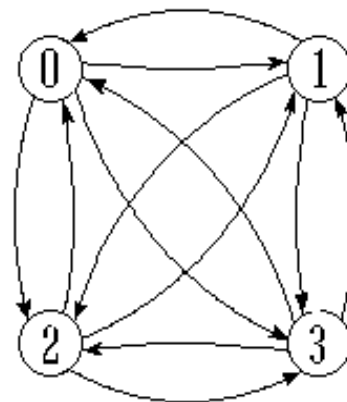
(b) G2

无向图(树)



(c) G3

有向图



(d) G4

有向 完全图

$n(n-1)/2$ 条边

$n(n-1)$ 条边

G1的顶点集合为 $V(G1)=\{0,1,2,3\}$

边集合为 $E(G1)=\{(0,1),(0,2),(0,3),(1,2),(1,3),(2,3)\}$

7.1 图的定义和术语

路径： 在图 $G = (V, E)$ 中, 若从顶点 v_i 出发, 沿一些边经过一些顶点 $v_{p1}, v_{p2}, \dots, v_{pm}$, 到达顶点 v_j 。则称顶点序列 $(v_i, v_{p1}, v_{p2}, \dots, v_{pm}, v_j)$ 为从顶点 v_i 到顶点 v_j 的**路径**。它经过的边 $(v_i, v_{p1}), (v_{p1}, v_{p2}), \dots, (v_{pm}, v_j)$ 应当是属于 E 的边。

路径长度： $\left\{ \begin{array}{l} \text{非带权图的路径长度是指此路径上边的条数;} \\ \text{带权图的路径长度是指路径上各边的权之和。} \end{array} \right.$

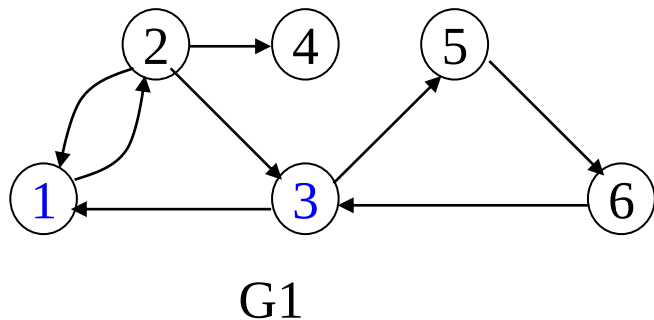
简单路径： 路径上各顶点 $v_1, v_2, \dots, v_m$ 均不互相重复。

回路： 若路径上第一个顶点 v_1 与最后一个顶点 v_m 重合, 则称这样的路径为**回路或环**。

简单回路： 图的顶点序列中, 除了第一个顶点和最后一个顶点相同外, 其余顶点不重复出现的回路叫简单回路。

7.1 图的定义和术语

例



路径 $1 \rightarrow 3$: **1, 2, 3** | (1, 2, 3, 5, 6, 3)

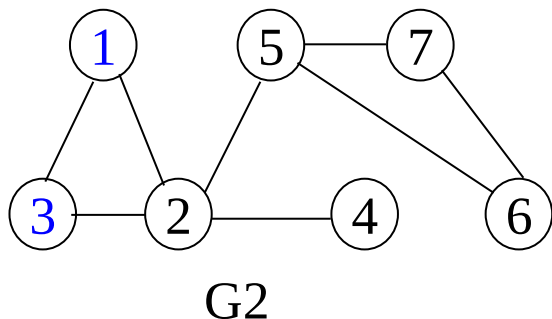
路径长度: 2 (5)

简单路径: **1, 2, 3, 5**

回路: 1, 2, 3, 5, 6, 3, 1

简单回路: 3, 5, 6, 3

例



路径: 1, 2, 5, 7, 6, 5, 2, 3

路径长度: 7

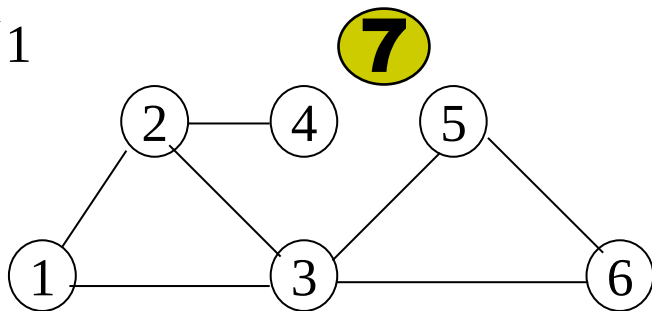
简单路径: 1, 2, 5, 7, 6

回路: 1, 2, 5, 7, 6, 5, 2, 1

简单回路: 1, 2, 3, 1

7.1 图的定义和术语

例1

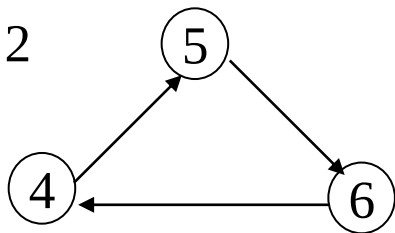


连通图

在**无向**图中, 若从顶点 v_i 到顶点 v_j 有路径, 则称顶点 v_i 与 v_j 是**连通**的。如果图中任意一对顶点都是连通的, 则称此图是**连通图**。

非连通图的**极大**连通子图叫做**连通分量**。

例2

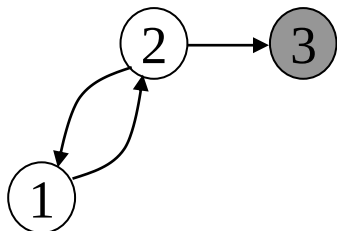


强连通图

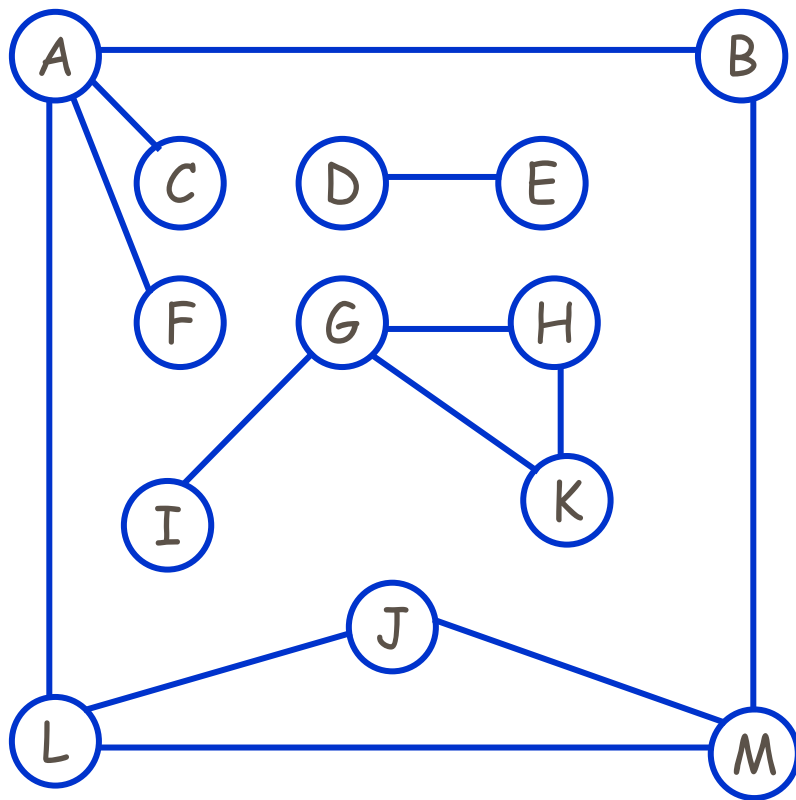
在**有向**图中, 若对于每一对顶点 v_i 和 v_j , 都存在一条从 v_i 到 v_j 和从 v_j 到 v_i 的路径, 则称此图是**强连通图**。

非强连通图的**极大**强连通子图叫做**强连通分量**。

例3



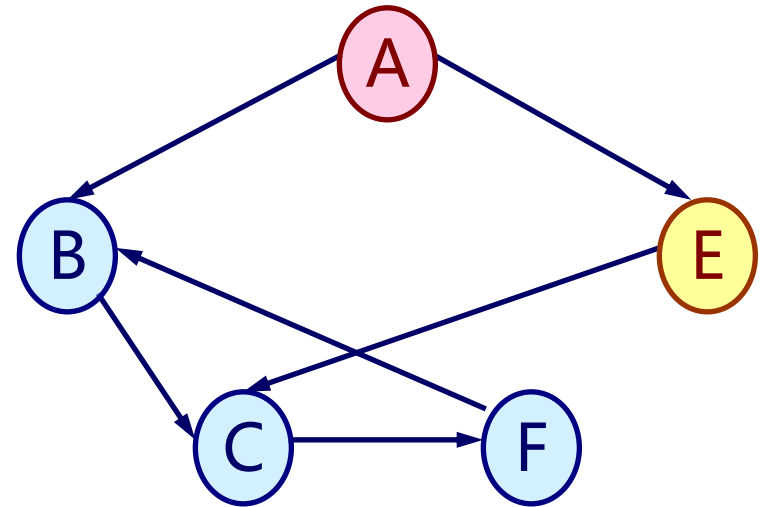
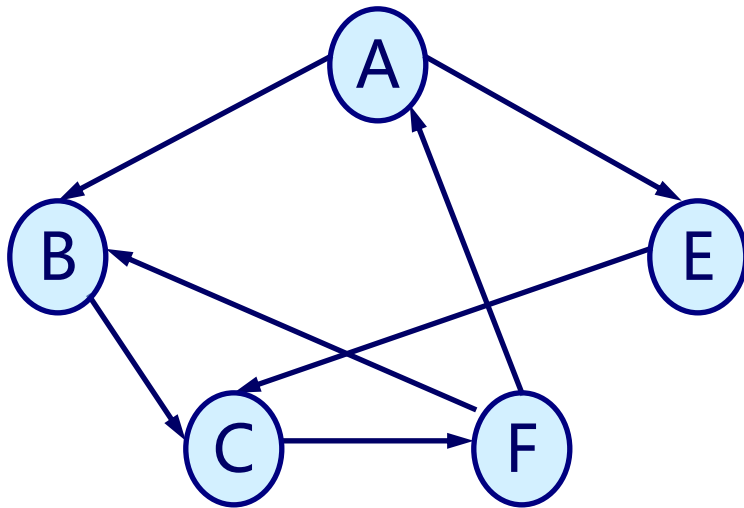
7.1 图的定义和术语



所谓**极大**是指连通分量中顶点数最多，再添加任何一个顶点就会使之变为非连通图。

若无向图为非连通图，则图中各个极大连通子图称作此图的**连通分量**。

7.1 图的定义和术语

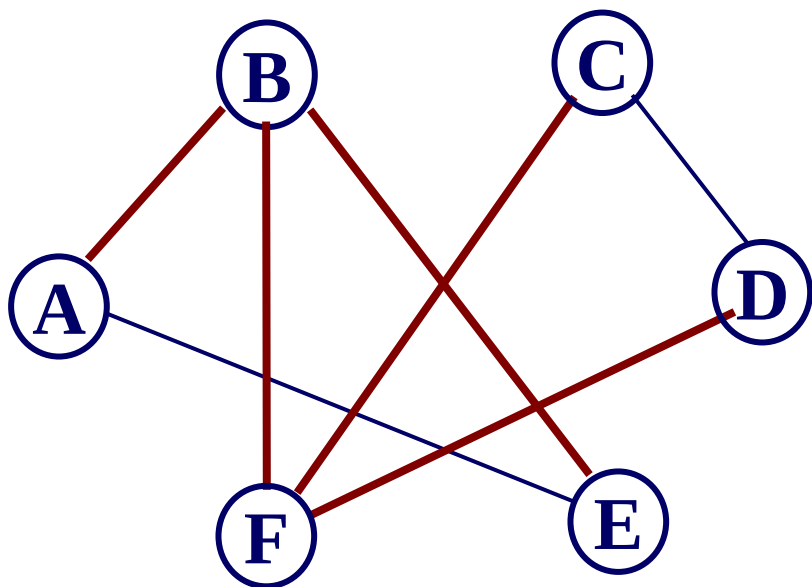


非强连通图的极大强连通子图叫做强连通分量

7.1 图的定义和术语

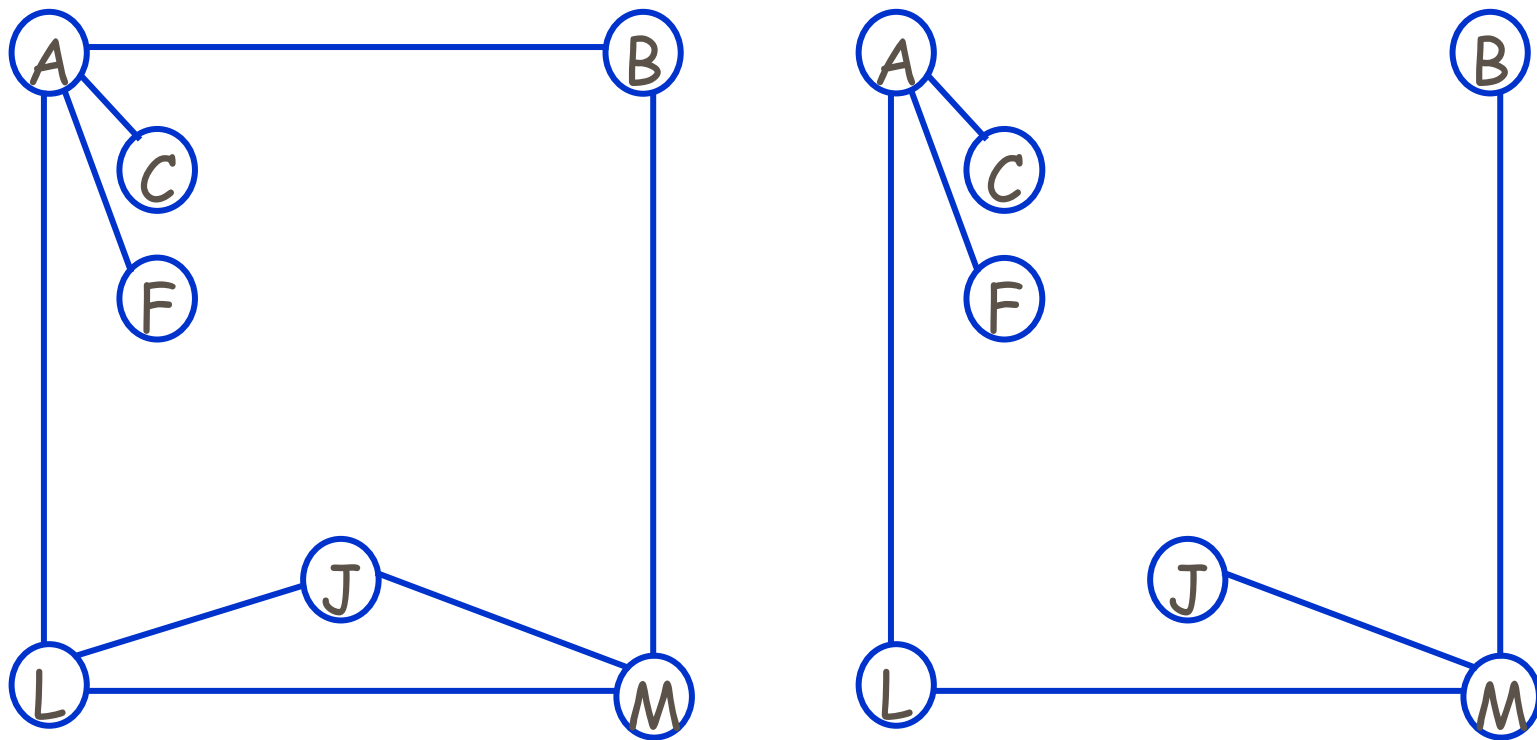
生成树

假设一个（无向）连通图有 n 个顶点和 e 条边，其中 $n-1$ 条边和 n 个顶点构成一个极小连通子图，称该极小连通子图为此连通图的**生成树**。



对非连通图，则称由各个连通分量的生成树的集合为此非连通图的**生成森林**。

7.1 图的定义和术语



所谓极小是指连通分量中边数最少，若在生成树中去掉任何一条边都会使之变为非连通图，若在生成数上任意添加一条边，就必会出现回路。

7.1 图的定义和术语

ADT Graph {

数据对象V: V是具有相同特性的数据元素的集合, 称为顶点集。

数据关系 R: $R=\{VR\}$; $VR=\{<v,w> | v,w \in V \text{ 且 } P(v,w),$
 $<v,w>$ 表示从v到w的弧,
 谓词 $P(v,w)$ 定义了弧 $<v,w>$ 的意义或信息}

基本操作P:

CreatGraph (&G, V,VR);

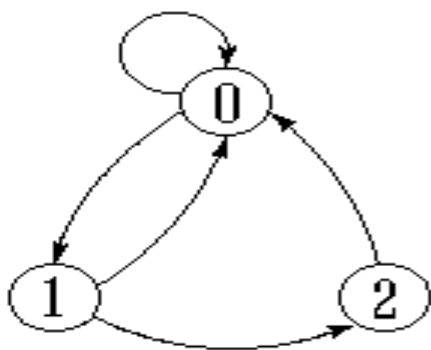
InsertVex (&G, v);

..... (参见教材P156-157)

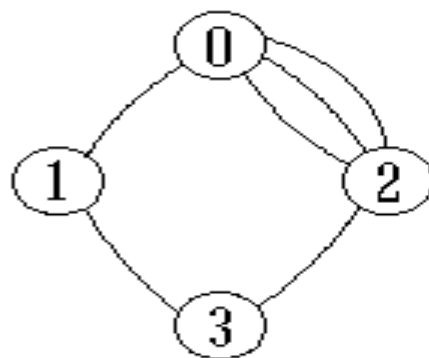
}ADT Graph

7.1 图的定义和术语

约定：不考虑顶点到其自身的边，也不允许一条边在图中重复出现(即若 (V_i, V_j) 或 $\langle V_i, V_j \rangle$ 是 $E(G)$ 中的一条边，则要求 $V_i \neq V_j$)，

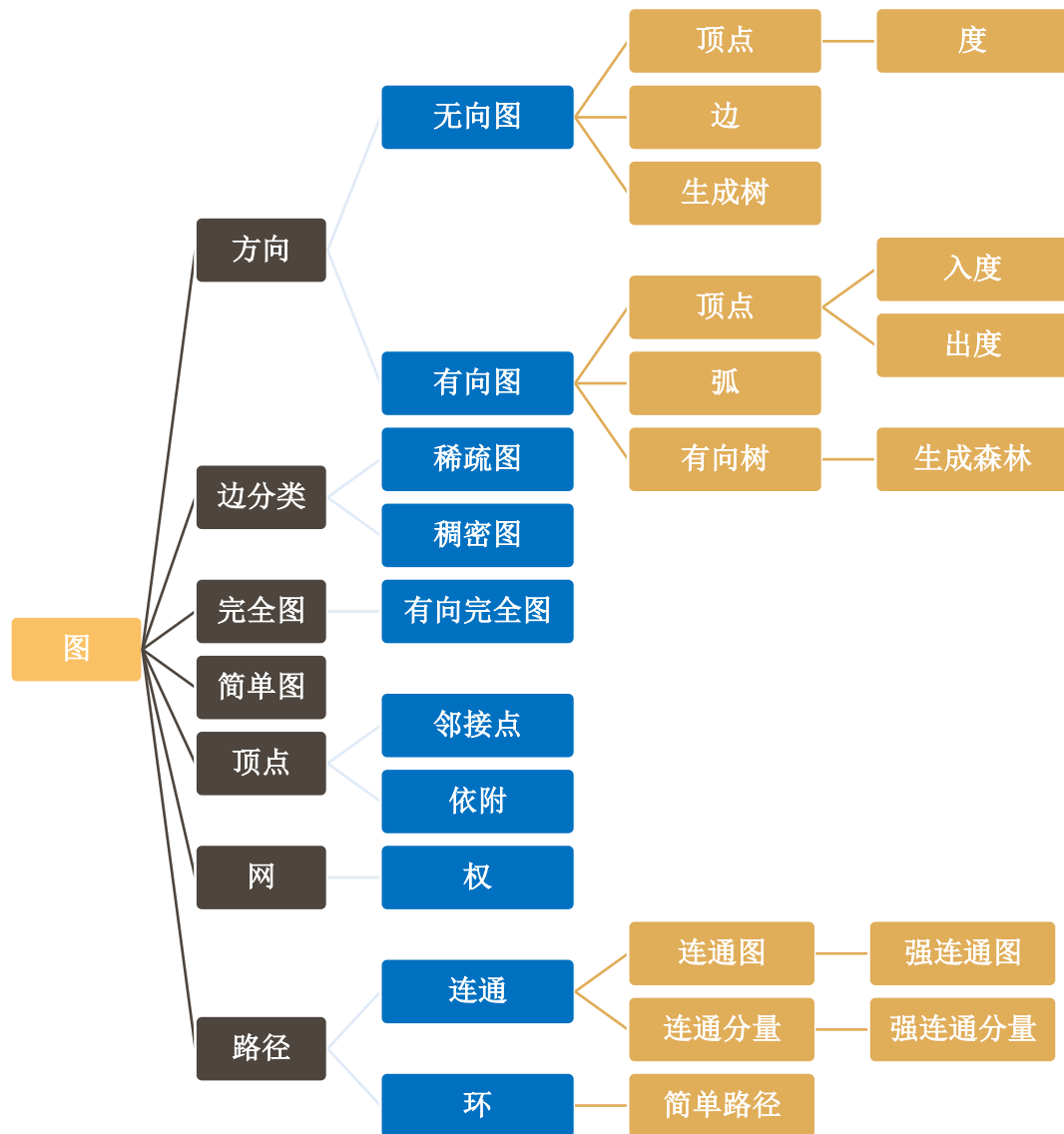


(a) 带自身环的图



(b) 多重图

7.1 图的定义和术语



第7章 图

7.1 图的定义和术语

7.2 图的存储结构

7.3 图的遍历

7.4 最小生成树

7.5 活动网络

7.6 最短路径

7.2 图的存储结构

顺序存储结构: **数组表示法 (邻接矩阵)**

链式存储结构: **多重链表**



邻接表
邻接多重表
十字链表

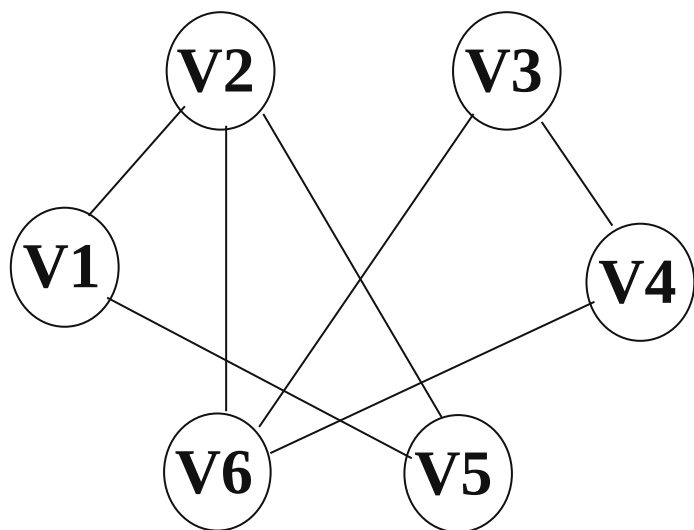
重点介绍: 邻接矩阵(数组)表示法
 邻接表(链式)表示法

7.2.1 数组（邻接矩阵）表示法

- **邻接矩阵**是表示顶点之间相邻关系的矩阵。设 $G=(V,VR)$ 是具有 $n(n>0)$ 个顶点的图，顶点的顺序依次为 $(v_0, v_1, \dots, v_{n-1})$ ，则 G 的邻接矩阵 A 是 n 阶方阵，其定义如下：
 - ◆ 如果 G 是**无向图**，则：
$$A[i][j] = \begin{cases} 1: \text{若 } (v_i, v_j) \in VR \\ 0: \text{其他} \end{cases}$$
 - ◆ 如果 G 是**有向图**，则：
$$A[i][j] = \begin{cases} 1: \langle v_i, v_j \rangle \in VR \\ 0: \text{其他} \end{cases}$$
 - ◆ 如果 G 是**带权无向图**，则：
$$A[i][j] = \begin{cases} w_{ij} : \text{若 } v_i \neq v_j \text{ 且 } (v_i, v_j) \in VR \\ \infty: \text{其他} \end{cases}$$
 - ◆ 如果 G 是**带权有向图**，则：
$$A[i][j] = \begin{cases} w_{ij} : \text{若 } v_i \neq v_j \text{ 且 } \langle v_i, v_j \rangle \in VR \\ \infty: \text{其他} \end{cases}$$

7.2.1 数组（邻接矩阵）表示法

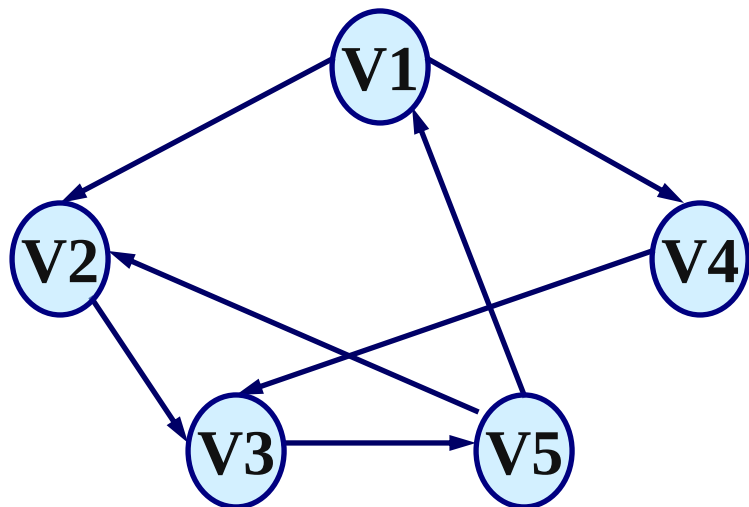
无向图



	1	2	3	4	5	6
1	0	1	0	0	1	0
2	1	0	0	0	1	1
3	0	0	0	1	0	1
4	0	0	1	0	0	1
5	1	1	0	0	0	0
6	0	1	1	1	0	0

- 无向图的邻接矩阵是**对称**的；
- 顶点*i* 的度=第 *i* 行 (列) 中**1** 的个数；
- 完全图的邻接矩阵中，对角元素为**0**，其余**1**

7.2.1 数组（邻接矩阵）表示法



	1	2	3	4	5	
1	0	1	0	1	0	
2	0	0	1	0	0	出度
3	0	0	0	0	1	
4	0	0	1	0	0	
5	1	1	0	0	0	入度

注：在有向图的邻接矩阵中，

第*i*行含义：以结点 v_i 为尾的弧(即出度边)；

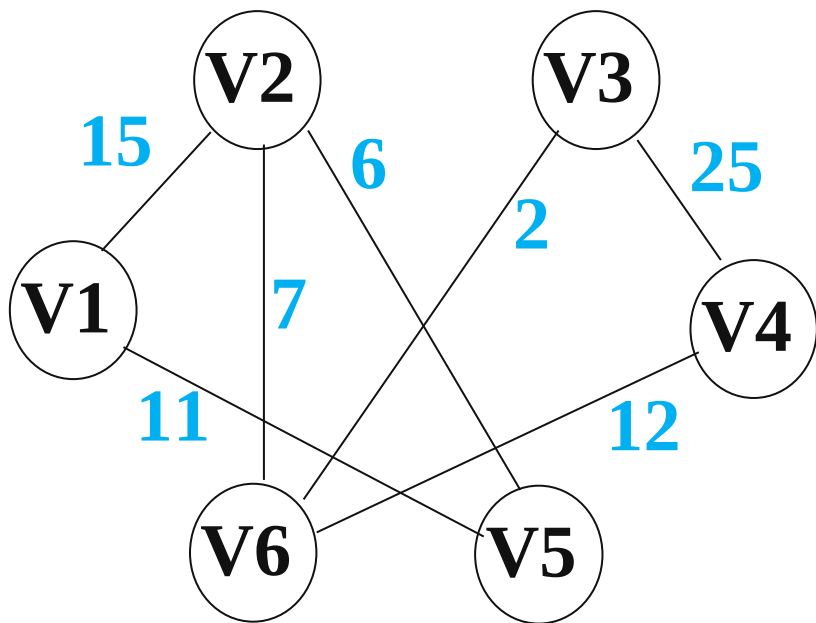
第*i*列含义：以结点 v_i 为头的弧(即入度边)。

- 有向图的邻接矩阵可能是不对称的。
- 顶点的出度=第*i*行元素之和
- 顶点的入度=第*i*列元素之和
- 顶点的度=第*i*行元素之和+第*i*列元素之和

7.2.1 数组（邻接矩阵）表示法

网的邻接矩阵可定义为： $A[i][j]=\begin{cases} w_{i,j}, (v_i, v_j) \in VR \\ \infty, (v_i, v_j) \notin VR \end{cases}$

无向网



	1	2	3	4	5	6
1	∞	15	∞	∞	11	∞
2	15	∞	∞	∞	6	7
3	∞	∞	∞	25	∞	2
4	∞	∞	25	∞	∞	12
5	11	6	∞	∞	∞	∞
6	∞	7	2	12	∞	∞

7.2.1 数组（邻接矩阵）表示法

□ 邻接矩阵的特点如下：

- 图的邻接矩阵表示是**惟一**的。
- 无向图的邻接矩阵一定是一个**对称矩阵**。因此，按照压缩存储的思想，在具体存放邻接矩阵时只需存放矩阵的上(或下)三角元素。
- 不带权的有向图的邻接矩阵一般来说是一个**稀疏矩阵**，当图的顶点较多时，可以采用三元组表的方法存储邻接矩阵。
- 对于无向图，邻接矩阵的第*i*行(或第*i*列)非零元素(或非 ∞ 元素)的个数正好是第*i*个顶点 v_i 的度。

7.2.1 数组（邻接矩阵）表示法

□ 邻接矩阵的特点如下：

- 对于有向图，邻接矩阵的第 i 行(或第 i 列)非零元素(或非 ∞ 元素)的个数正好是第 i 个顶点 v_i 的出度(或入度)。
- 用邻接矩阵方法存储图，容易实现图的操作，如：求某顶点的度、判断顶点之间是否有边、找顶点的邻接点等等。
- n 个顶点需要 $n*n$ 个单元存储边，空间效率为 $O(n^2)$ 。对稀疏图而言尤其浪费空间。要确定图中有多少条边，则必须按行、按列对每个元素进行检测，所花费的时间代价很大。这是用邻接矩阵存储图的局限性。

7.2.1 数组（邻接矩阵）表示法

```
typedef enum{DG,DN,UDG,UDN}GraphKind; //{有向图，有向网，无向图，无向网}
```

```
typedef struct ArcCell { // 弧的定义  
    VRType adj;          // VRType是顶点关系类型。  
    // 对无权图，用1或0表示相邻否；对带权图，则为权值类型。  
    InfoType *info;      // 该弧相关信息的指针  
} ArcCell, AdjMatrix[MAX_VERTEX_NUM][MAX_VERTEX_NUM]; // 二维数组
```

```
typedef struct { // 图的定义  
    VertexType vexs[MAX_VERTEX_NUM]; // 顶点向量，顶点信息  
    AdjMatrix arcs;                  // 邻接矩阵，弧信息  
    int vexnum, arcnum;              // 顶点数，弧数  
    GraphKind kind;                  // 图的种类标志  
} MGraph;
```

7.2.1 数组（邻接矩阵）表示法

□ 采用邻接矩阵构造无向网

- 输入总顶点数和总边数。
- 依次输入点的信息存入顶点表中。
- 初始化邻接矩阵，使每个权值初始化为极大值。
- 构造邻接矩阵。

7.2.1 数组（邻接矩阵）表示法

```
Status CreateUDN (Mgraph &G) { //构造无向网
```

```
    scanf(&G.vexnum,&G.arcnum,&IncInfo); //输入顶点，边数
```

```
    for(i=0;i< G.vexnum;++i) scanf(&G.vexs[i]); //构造顶点向量
```

```
    for(i=0;i< G.vexnum;++i) //初始化邻接矩阵
```

```
        for(j=0;j< G.vexnum;++j) G.arcs[i][j]={INFINITY,NULL};
```

```
    for(k=0;k< G.arcnum;++k){ //构造邻接矩阵
```

```
        scanf(&v1,&v2, &w); //输入一条边依附的顶点
```

```
        i=LocateVex(G,v1); j= LocateVex(G,v2); //定位v1, v2
```

```
        G.arcs[i][j].adj=w; //弧< v1, v2 >的权值
```

```
        if(IncInfo) Input (*G.arcs[i][j].info); //输入边相关信息
```

```
        G.arcs[j][i]= G.arcs[i][j]; //置邻接矩阵对称值
```

```
    }
```

```
}
```

7.2.1 数组（邻接矩阵）表示法

```
int LocateVex(MGraph G,char u) {  
    int i;  
    for (i=0;i<G.vexnum;i++) {  
        if (u == G.vexs[i]) return i;  
    }  
    if (i== G.vexnum) {  
        printf("Error u!\n");  
        exit(1);  
    }  
    return 0;  
}
```

7.2.1 数组（邻接矩阵）表示法

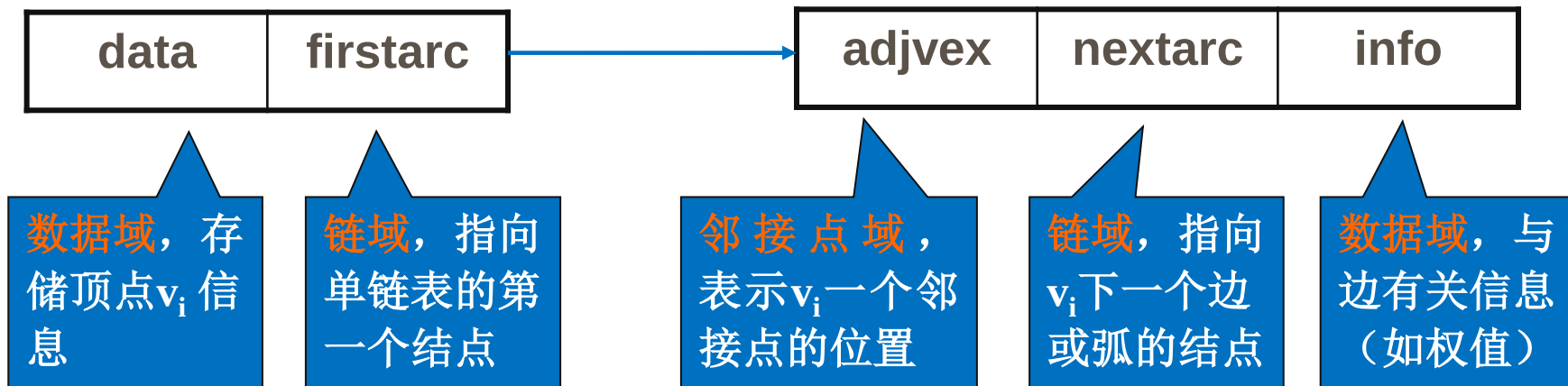
```
void PrintMGraph(MGraph G) {  
    int i, j;  
    printf("Output Vertices:");  
    printf("%s",G.vexs); printf("\n");  
    printf("Output AdjMatrix:\n");  
    for (i=0;i<G.vexnum;i++) {  
        for (j=0;j<G.vexnum;j++)  
            printf("%4d",G.arcs[i][j]);  
        printf("\n");  
    }  
    return;  
}
```

7.2.2 图的邻接表（链式）存储表示

- 对每个顶点 v_i 建立一个单链表，把与 v_i 有关联的边的信息链接起来，每个结点设为3个域；

头结点

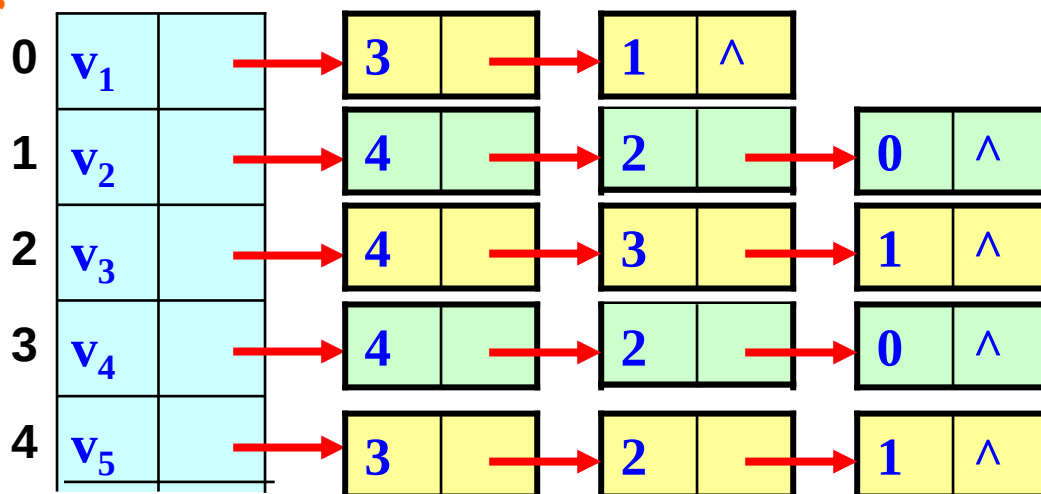
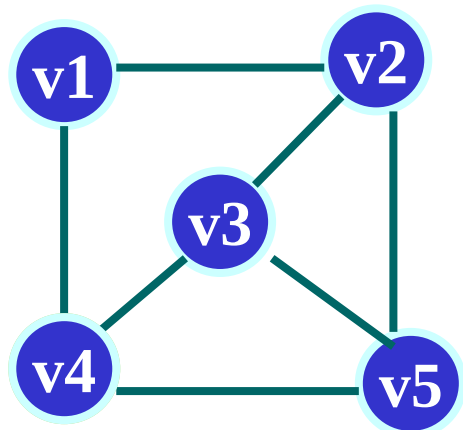
表结点



- 每个单链表有一个头结点（设为2个域），存 v_i 信息；
- 每个单链表的头结点通常以顺序存储结构存储。

7.2.2 图的邻接表（链式）存储表示

□ 无向图的邻接表表示



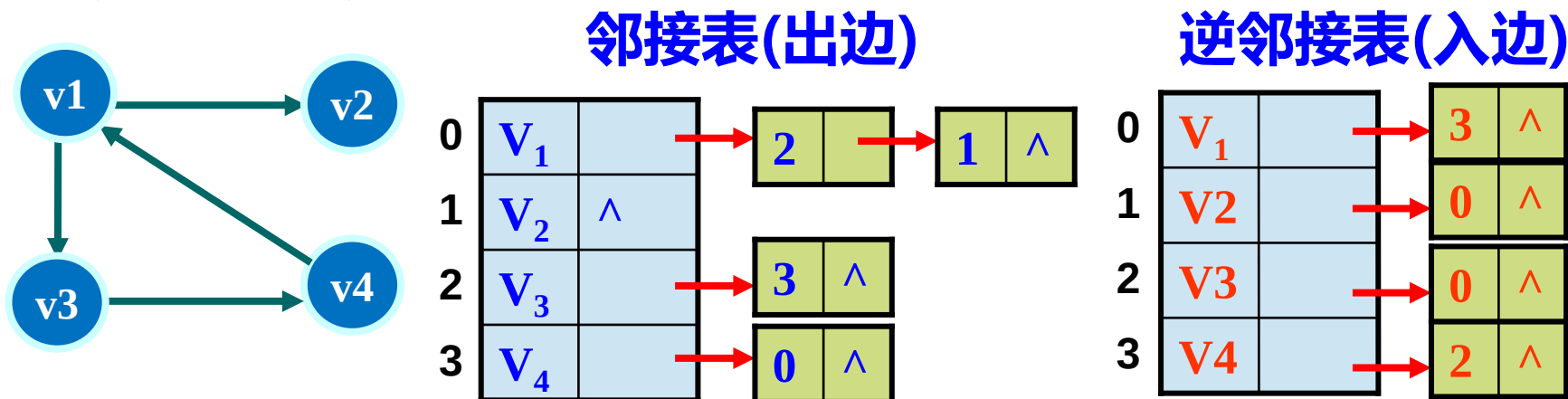
注：邻接表不唯一，因各个边结点的链入顺序是任意的

- 每条边以所依附的两个顶点的形式出现2次，空间效率为 $O(n+2e)$ 。
- 若是稀疏图 ($e \ll n^2$)，比邻接矩阵表示法 $O(n^2)$ 省空间。

TD(V_i)=单链表中链接的结点个数

7.2.2 图的邻接表（链式）存储表示

□ 有向图的邻接表表示



空间效率为 $O(n+e)$

出度: $OD(V_i) =$ 单链出边表中链接的结点数

入度: $ID(V_i) =$ 邻接点域的值为 i 的结点个数

度: $TD(V_i) = OD(V_i) + ID(V_i)$

7.2.2 图的邻接表（链式）存储表示

□ 图的邻接表存储定义

- 弧的结点结构
- 顶点的结点结构
- 图的结构定义

弧的结点结构

adjvex	nextarc	info
--------	---------	------

```
typedef struct ArcNode {  
    int      adjvex; // 该弧所指向的顶点的位置  
    struct ArcNode *nextarc; // 指向下一条弧的指针  
    InfoType *info; // 该弧相关信息的指针  
} ArcNode;
```

7.2.2 图的邻接表（链式）存储表示

□ 图的邻接表存储定义

- 弧的结点结构
- 顶点的结点结构
- 图的结构定义

顶点的结点结构

data	firstarc
------	----------

```
typedef struct VNode {  
    VertexType data; // 顶点信息  
    ArcNode *firstarc;  
    // 指向第一条依附该顶点的弧  
} VNode, AdjList[MAX_VERTEX_NUM];
```

7.2.2 图的邻接表（链式）存储表示

□ 图的邻接表存储定义

- 弧的结点结构
- 顶点的结点结构
- 图的结构定义

图的结构

```
typedef struct {  
    AdjList vertices; //顶点向量  
    int vexnum, arcnum; // 图的当前顶点数和弧数  
    int kind; // 图的种类标志  
} ALGraph;
```

7.2.2 图的邻接表（链式）存储表示

构造有向图的邻接表算法

- 输入顶点数和弧数
- 输入顶点表头向量

输入弧 $\langle v_1, v_2 \rangle$

$\langle A, B \rangle$

定位弧的顶点 v_1 和 v_2

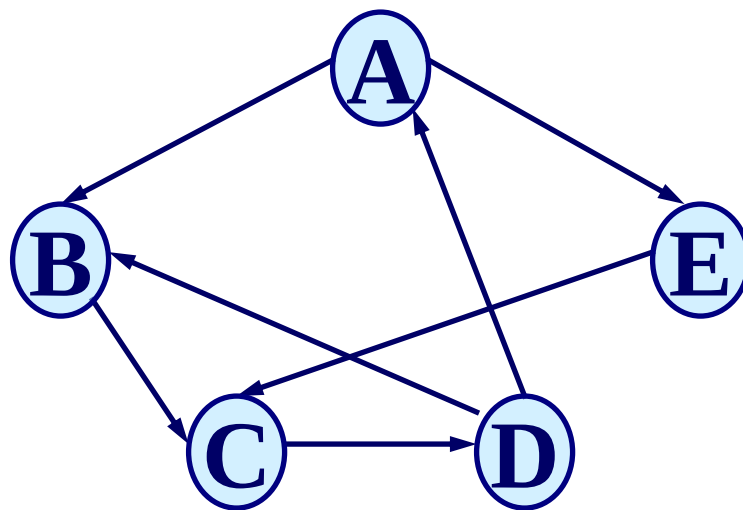
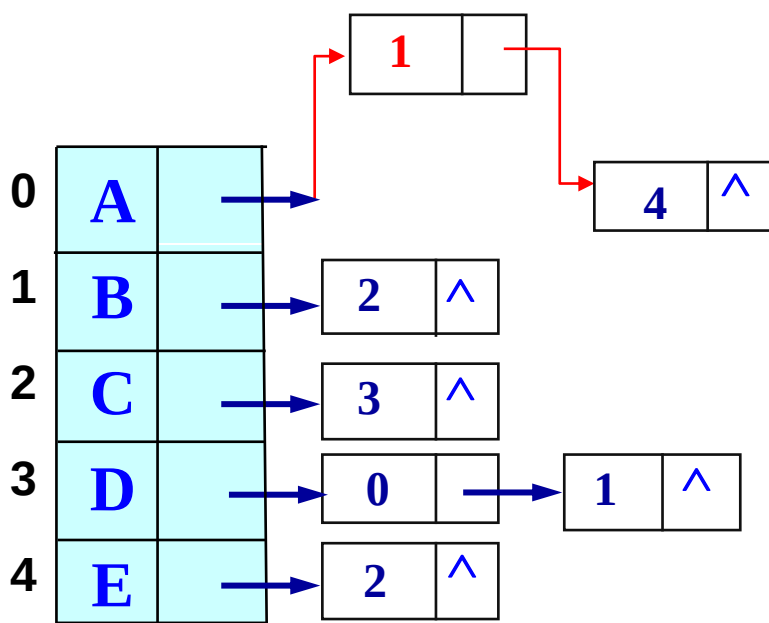
0, 1

产生一个新的弧结点

对新弧结点赋值

1

插入弧结点到单链表



7.2.2 图的邻接表（链式）存储表示

□ 构造有向图的邻接表算法

```
Status CreateDG ( ALGraph &G ) {  
    scanf ( &G.vexnum, &G.arcnum );    //输入顶点和弧数  
    for ( i = 0; i < G.vexnum; ++i ) {    //初始化表头向量  
        scanf ( &G.vertices[i].data);  G.vertices[i].firstarc=NULL ; }  
    for ( k = 0; k < G.arcnum; ++k ) {    //构造邻接表  
        scanf ( &v1, &v2 );                //输入各弧  
        i = LocateVex ( G, v1 ); j = LocateVex ( G, v2 );  
        if ( !p=(ArcNode*)malloc(sizeof(ArcNode)))  
            exit(OVERFLOW)    //产生一个新的弧结点  
        p.adjvex = j;          // 对弧结点赋值  
        p.info = NULL;  
        p.nextarc= G.vertices[i].firstarc; //插入弧结点到单链表  
        G.vertices[i].firstarc= p; }      //头插法  
} // CreateDG
```

时间复杂度: $O(e \cdot n)$

7.2.2 图的邻接表（链式）存储表示

□ 邻接表表示法的特点

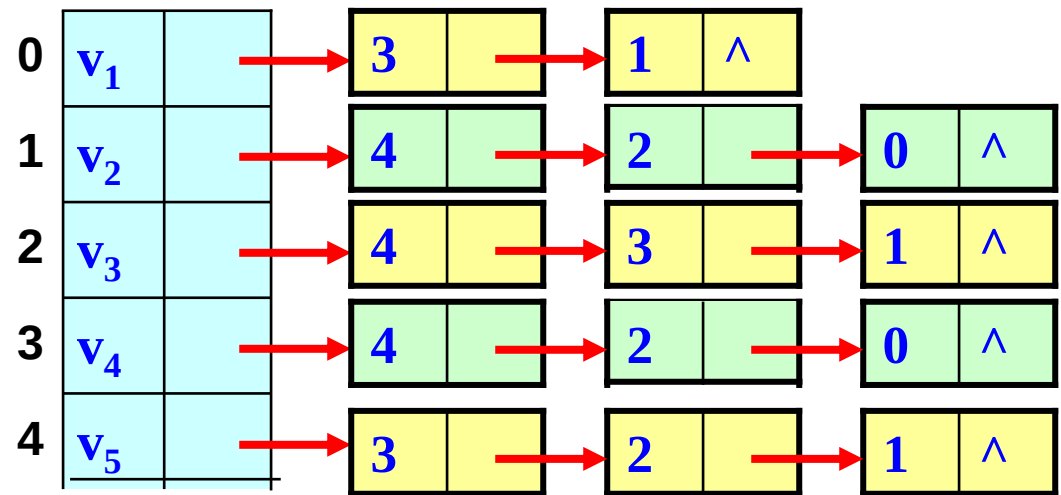
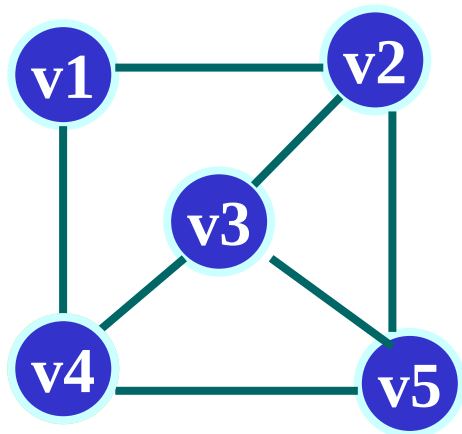
- **优点：** 空间效率高，容易寻找顶点的邻接点；
- **缺点：** 判断两顶点间是否有边或弧，需搜索两结点对应的单链表，没有邻接矩阵方便。

7.2.2 图的邻接表（链式）存储表示

```
void CreateALGraph_adjlist(ALGraph &G){ //带权邻接表
    int i, j, k, w; char v1, v2; ArcNode *p;
    printf("Input vexnum & arcnum:");
    scanf("%d,%d", &G.vexnum, &G.arcnum);
    printf("Input Vertices:");
    for (i = 0; i < G.vexnum; i++){
        scanf("%c", &G.vertices[i].data);
        G.vertices[i].firstarc = NULL;
    }
    printf("Input Arcs(v1,v2 & w):\n");
    for (k = 0; k < G.arcnum; k++){
        scanf("%c,%c,%d", &v1, &v2, &w);
        i = LocateVex(G, v1);
        j = LocateVex(G, v2);
        p = (ArcNode *)malloc(sizeof(ArcNode));
        p->adjvex = j;
        p->info = w;
        p->nextarc = G.vertices[i].firstarc;
        G.vertices[i].firstarc = p;
    }
    return;
}
```

```
// p=(ArcNode*)malloc(sizeof(ArcNode));
// p->adjvex=i; p->info = w;
// p->nextarc=G.vertices[j].firstarc;
// G.vertices[j].firstarc=p;
```


7.2.3 邻接矩阵与邻接表表示法的关系

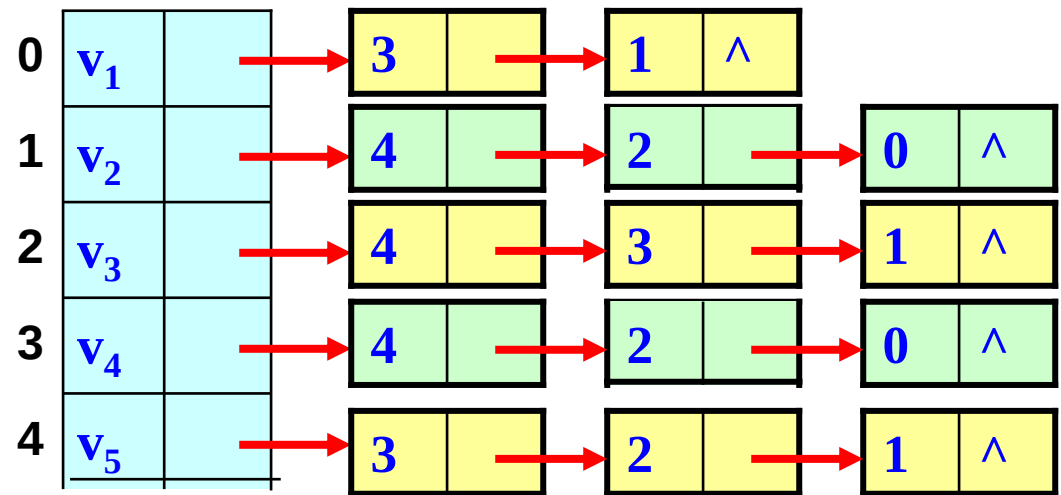
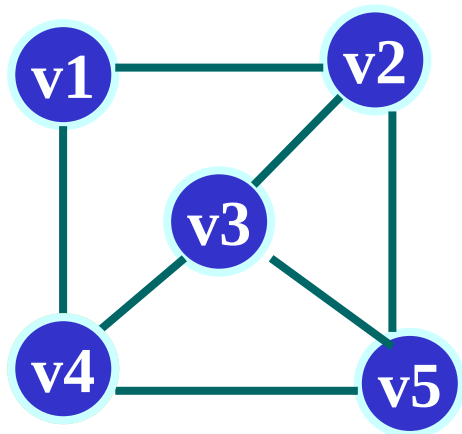


(v1 v2 v3 v4 v5)

v1	0	1	0	1	0
v2	1	0	1	0	1
v3	0	1	0	1	1
v4	1	0	1	0	1
v5	0	1	1	1	0

- **联系：** 邻接表中每个链表对应于邻接矩阵中的一行，链表中结点个数等于一行中非零元素的个数。

7.2.3 邻接矩阵与邻接表表示法的关系

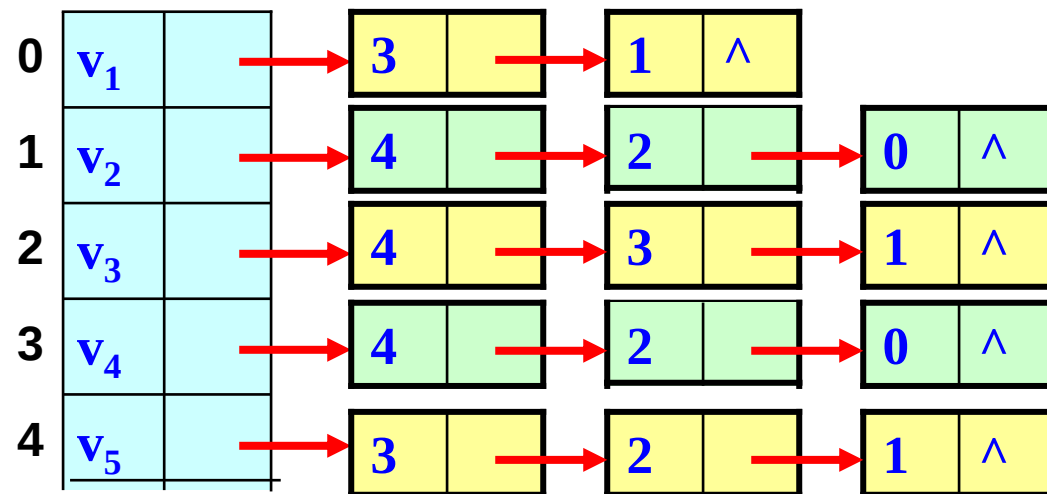
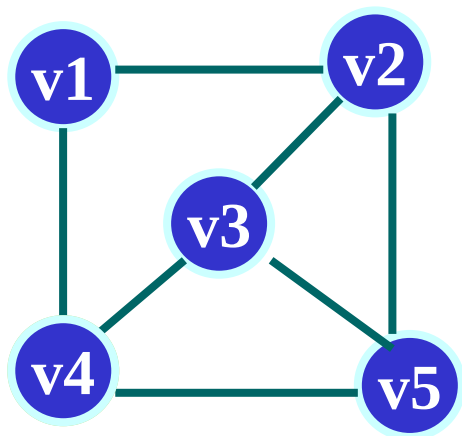


(v_1 v_2 v_3 v_4 v_5)

v_1	0	1	0	1	0
v_2	1	0	1	0	1
v_3	0	1	0	1	1
v_4	1	0	1	0	1
v_5	0	1	1	1	0

- 区别：
 - ① 对于任一确定的无向图，邻接矩阵是**唯一**的（行列号与顶点编号一致），但邻接表**不唯一**（链接次序与顶点编号无关）。
 - ② 邻接矩阵的空间复杂度为 $O(n^2)$ ，而邻接表的空间复杂度为 $O(n+e)$ 。

7.2.3 邻接矩阵与邻接表表示法的关系



(v_1 v_2 v_3 v_4 v_5)

	v_1	v_2	v_3	v_4	v_5
v_1	0	1	0	1	0
v_2	1	0	1	0	1
v_3	0	1	0	1	1
v_4	1	0	1	0	1
v_5	0	1	1	1	0

- 用途：邻接矩阵多用于稠密图；而邻接表多用于稀疏图。

7.2.4 十字链表表示法

它是有向图的另一种链式结构。

思路：将邻接矩阵用链表存储，是邻接表、逆邻接表的结合。

- 1、开设弧结点，设5个域（每段弧是一个数据元素）
- 2、开设顶点结点，设3个域（每个顶点也是一个数据元素）

7.2.4 十字链表表示法

弧结点

tailvex	headvex	hlink	tlink	info
---------	---------	-------	-------	------



tailvex: 弧尾顶点位置

headvex: 弧头顶点位置

hlink: 弧头相同的下一弧位置

tlink: 弧尾相同的下一弧位置

info: 弧信息

顶点结点

data	Firstin	Firstout
------	---------	----------



data : 顶点信息

Firstin : 以顶点为弧头的第一条弧
结点

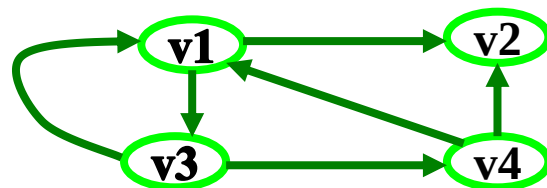
Firstout: 以顶点为弧尾的第一条弧
结点

n个顶点的集合怎样储存?



仍用顺序存储结构

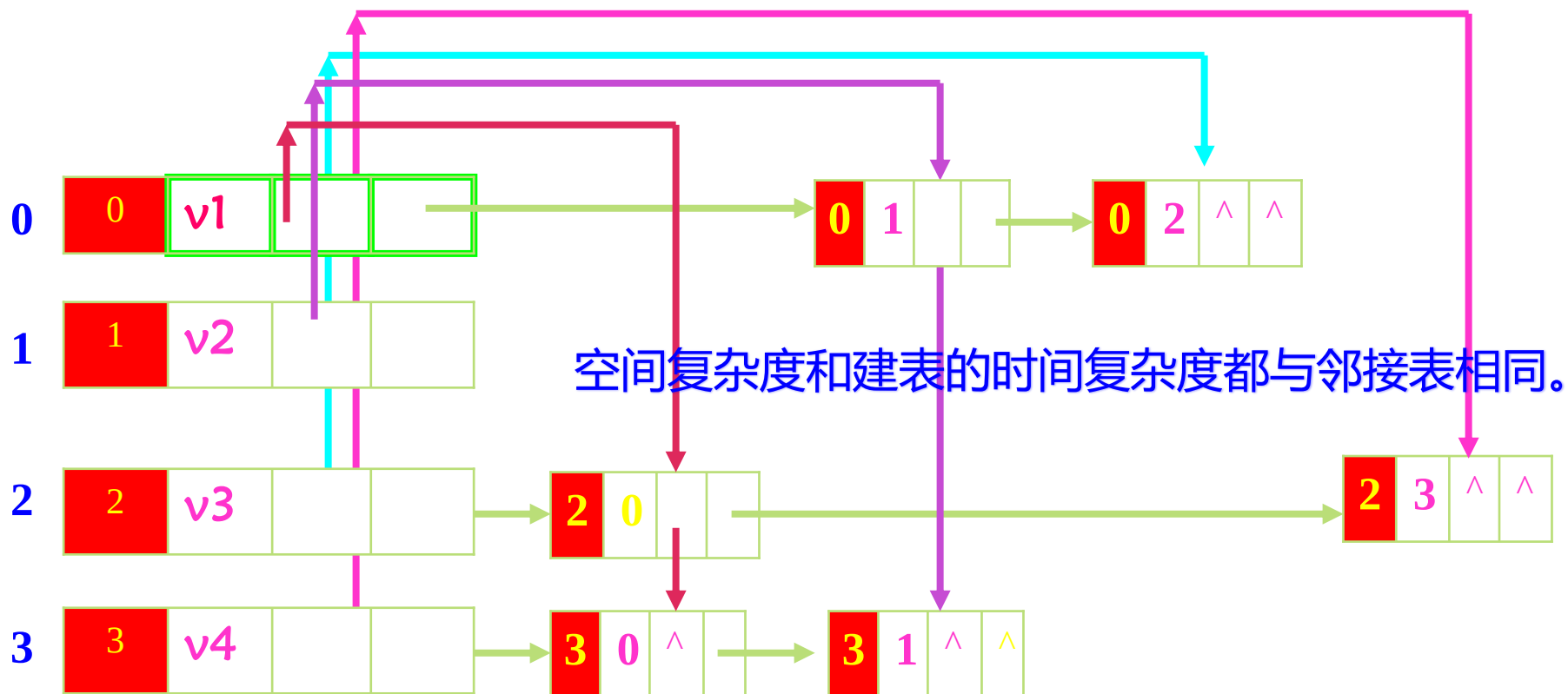
7.2.4 十字链表表示法



顶点结点	data	Firstin	Firstout
------	------	---------	----------

弧结点	tailvex	headvex	hlink	tlink	info
-----	---------	---------	-------	-------	------

此无权图未
开第4分量



十字链表优点：容易操作，如求顶点的入度、出度等。

7.2.4 十字链表表示法

```
#define MAX_VERTEX_NUM 20
```

```
typedef struct ArcBox {    //弧结点结构, 5分量  
    int tailvex, headvex;  
    struct ArcBox * hlink, * tlink;  
    InfoType *info;  
} ArcBox;
```

```
typedef struct VexNode{ //顶点结构, 3分量  
    VertexType data;  
    ArcBox * firstin,*firstout;  
}VexNode;
```

```
typedef struct {           //图结构,整体概念  
    VexNode xlist[ MAX_VERTEX_NUM ]; //表头向量  
    int vexnum, arcnum;  
}OLGraph;
```

7.2.4 十字链表表示法

```
Status CreateDG(OLGraph &G) { //采用十字链表存储表示, 构造有向图
    scanf(&G.vexnum, &G.arcnum, &IncInfo);
    for(i = 0; i<G.vexnum; ++i) { //构造表头向量
        scanf(&G.xlist[i].data); //输入顶点值
        G.xlist[i].firstin = NULL;
        G.xlist[i].firstout = NULL; //初始化指针
    }
    for(k=0; k<G.arcnum; ++k) { //输入各弧并构造十字链表
        scanf(&v1, &v2); //输入一条弧的起点和终点
        i = LocateVex(G, v1); j = LocateVex(G, v2);
        //确定v1和v2在G中的位置
        p = (ArcBox *)malloc(sizeof(ArcBox)); //为弧结点分配空间
        *p = {i,j,G.xlist[j].firstin,G.xlist[i].firstout,NULL}
        //对弧结点赋值
        G.xlist[j].firstin = G.xlist[i].firstout = p;
        //完成在入弧和出弧链头的插入
        if(IncInfo) Input(*p ->info); //若弧含有相关信息, 则输入
    }
} //CreateDG
```


7.2.5 邻接多重表表示法

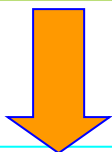
这是无向图的另一种存储结构，当对边操作时建议采用此种结构存储。

- 1、设立边结点，6个域（每条边是一个数据元素）
- 2、设立顶点结点，2个域（每个顶点也是一个数据元素）

7.2.5 邻接多重表表示法

边结点

mark	ivex	ilink	ivex	jlink	info
------	------	-------	------	-------	------



mark: 标志域，是否搜索过
ivex, jvex: 边依附的两个顶点位置
ilink: 指向下一条依附顶点 i 的边位置
jlink: 指向下一条依附顶点 j 的边位置
info: 边信息

顶点结点

data	Firstedge
------	-----------

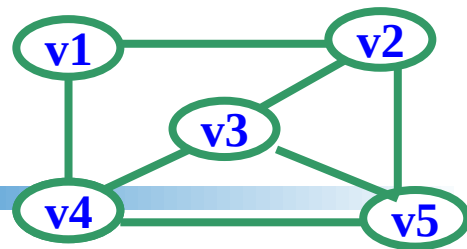


data : 存储顶点信息
Firstedge: 依附顶点的第一条边结点

n个顶点的集合怎样储存?

仍用顺序存储结构

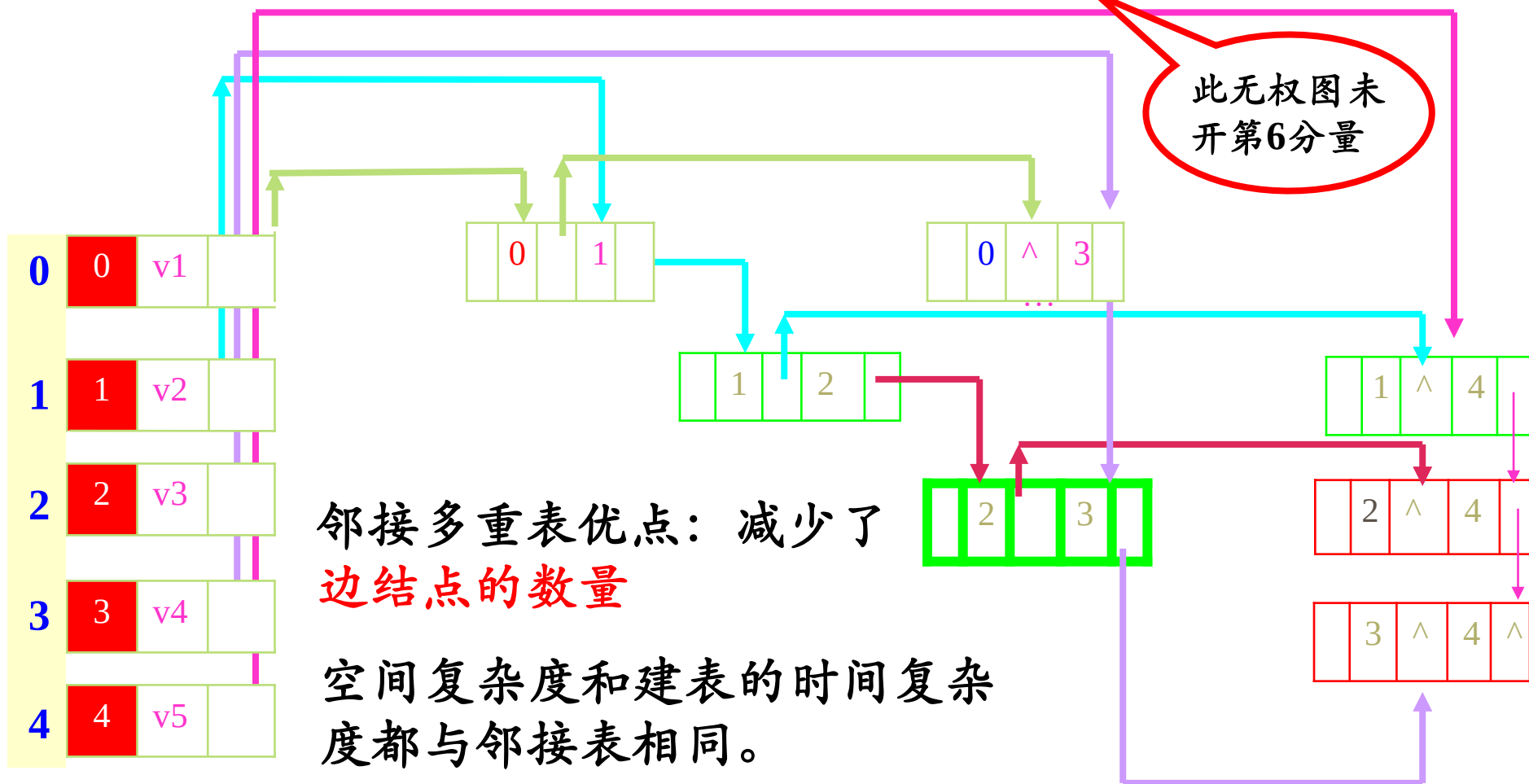
7.2.5 邻接多重表表示法



顶点结点

边结点

data	Firstedge	mark	ivex	ilink	ivex	jvex	jlink	info
------	-----------	------	------	-------	------	------	-------	------



7.2 图的存储结构

各种存储结构的适用类型

- 数组(邻接矩阵): 有向图和无向图
- 邻接表（逆邻接表）: 有向图和无向图
- 十字链表: 有向图
- 邻接多重表: 无向图

正在答疑
